

✓ Empty-Shelf Detector — Week 1

Fine-tune YOLO11 to flag out-of-stocks on retail shelf photos.

✓ 0. Turn the GPU on first

Colab gives you a free GPU but **not by default**. Do this once:

Runtime → Change runtime type → Hardware accelerator → T4 GPU → Save.

Then run the cell below. If it prints a table with "Tesla T4", you're set. If it errors, the GPU isn't on yet.

```
!nvidia-smi
```

✓ 1. Install

`ultralytics` is the YOLO library. `roboflow` is just to pull the dataset.

```
!pip install ultralytics roboflow -q

import ultralytics
ultralytics.checks() # prints versions + confirms the GPU is visible to YOLO
from ultralytics import YOLO
```

✓ 2. Get the data

We use a ready-labeled empty-shelf dataset from **Roboflow Universe** so you skip annotation entirely this week. A good candidate:

- `fyp-ormnr/supermarket-empty-shelf-detector` (≈500 images, labels empty / out-of-stock regions)

Other options if that one's down: `empty-space-detection-capstone/out-of-stock-detection`, or browse <https://universe.roboflow.com/browse/retail>

How to get YOUR download snippet (60 seconds)

1. Open the dataset page on Roboflow Universe.
2. Click **Download Dataset** → format **YOLOv11** → **show download code**.
3. It hands you a snippet with *your* API key + the exact workspace/project/version filled in.
4. Paste that over the placeholder cell below.

A free Roboflow account gives you the API key. The snippet looks like the template below — replace the three ALL-CAPS values (or just paste Roboflow's generated version verbatim).

```
# — Replace with the snippet from Roboflow's "show download code" —
# !pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key="YOUR_API_KEY")
project = rf.workspace("fyp-ormnr").project("supermarket-empty-shelf-detector")
version = project.version(4)
dataset = version.download("yolov11")

# _____

print("Downloaded to:", dataset.location)
```

✓ Quick fix Roboflow's data.yaml (do this — it saves an annoying error)

Roboflow writes relative paths into `data.yaml` that sometimes break in Colab. This rewrites them to absolute paths so training just works.

```
import yaml, os

base = "/content/Supermarket-Empty-Shelf-Detector--4" # the path from your error
yaml_path = os.path.join(base, "data.yaml")

with open(yaml_path) as f:
    d = yaml.safe_load(f)

def find_images(base, names):
    for n in names:
        p = os.path.join(base, n, "images")
        if os.path.isdir(p):
            return p
    return None

d["train"] = find_images(base, ["train"])
d["val"] = find_images(base, ["valid", "val"]) # Roboflow uses "valid"
test = find_images(base, ["test"])
if test:
    d["test"] = test

with open(yaml_path, "w") as f:
    yaml.safe_dump(d, f)

print("Classes:", d.get("names"))
print("train:", d["train"])
print("val: ", d["val"])
```

3. Fine-tune YOLO11

`yolo11n` = the nano model: smallest and fastest, perfect for a free T4 and a small dataset. (`yolo11s`/`yolo11m`) are bigger and more accurate but slower — try them later.)

You're not training from scratch — you start from weights pretrained on millions of images and *fine-tune* on shelves. That's why ~50 epochs on a few hundred images is enough to get something real.

Why YOLO11 and not the newer YOLO26? YOLO26 shipped Jan 2026 and is great, but YOLO11 has a year of tutorials, forum answers, and Stack Overflow threads behind it. When you hit an error at 11pm, you want the version everyone else has already debugged. Switching later is literally one word: `yolo26n.pt`. Learn on the well-worn path.

```
# model = YOLO("/content/drive/MyDrive/shelf_project/best.pt") # pretrained weights -> trained with Robotflow
model = YOLO("yolo11n.pt")
results = model.train(
    data=yaml_path,
    epochs=50,
    imgsz=640,
    batch=16,
    patience=15, # stop early if it plateaus
    name="shelf_v1",
)
```

4. How good is it?

`mAP50` is the headline number (mean average precision at IoU 0.5). Don't chase a specific value yet — just read it, eyeball the predictions next, and note it in your writeup. The training run also saved plots (confusion matrix, PR curve) into the run folder.

```
metrics = model.val()
print("mAP50: ", round(metrics.box.map50, 3))
print("mAP50-95: ", round(metrics.box.map, 3))
```

5. Run it on an image

This runs your trained model on validation images and shows the boxes. Later, swap in a photo *you* took at a real store — that's step 7.

```
import glob
from PIL import Image
from IPython.display import display

val_images = glob.glob(os.path.join(dataset.location, "valid", "images", "*"))
val_images = val_images or glob.glob(os.path.join(dataset.location, "val", "images", "*"))

preds = model.predict(val_images[:3], save=True, conf=0.25)
for p in preds:
    display(Image.fromarray(p.plot()[..., ::-1])) # BGR→RGB
```

6. Turn boxes into a number a brand cares about ← your edge

This is the part most CV people skip and you won't, because you lived it at P&G. The model gives boxes; *you* know which metric a brand manager actually opens the email for.

Below: count detections per class and compute a simple **on-shelf availability** proxy. Check `model.names` first and set which class label means "empty" for your dataset (it might be `empty`, `Out-of-Stock`, `gap`, etc.).

```
print("Class names in this model:", model.names)

# Δ set this to whichever label means an empty/out-of-stock region
EMPTY_LABELS = {"empty", "Empty", "Out-of-Stock", "out-of-stock", "gap", '100- 0-0-S'}

def shelf_report(image_path, conf=0.25):
    r = model.predict(image_path, conf=conf, verbose=False)[0]
    h, w = r.orig_shape # image size in pixels
    n_gaps = len(r.bboxes) # number of out-of-stock regions

    gap_area = sum(bw * bh for _, _, bw, bh in r.bboxes.xywh.tolist())
    gap_pct = gap_area / (h * w) * 100

    print(f"\n{os.path.basename(image_path)}")
    print(f" out-of-stock regions: {n_gaps}")
    print(f" empty area: {gap_pct:.1f}% of the image")
    return n_gaps, gap_pct

for img in val_images[:3]:
    shelf_report(img)
```

7. Save the model + ship it

Your trained weights are at `runs/detect/shelf_v1/weights/best.pt`. The cell downloads them.

Then — this is what makes it a real project, not a tutorial you watched:

1. Take 3–5 photos of an actual shelf at a local store (some full, some with gaps).
2. Run them through `shelf_report()` above.
3. Push to a GitHub repo: the notebook, a few result images, and a short README — what it does, the mAP50, the empty area metric, and one line connecting it to retail out-of-stocks (you know that pitch cold).

That repo is the artifact. It's your first CV project *and* the first brick of a domain-specific portfolio. Expansion comes after the ugly working version exists — not before.

```
from google.colab import files

# best.pt is the model. Later run with YOLO("best.pt")
candidates = glob.glob("/content/runs/detect/**/weights/best.pt", recursive=True)
print("Found:", candidates)

latest = max(candidates, key=os.path.getmtime) # the most recent run
print("Downloading:", latest)
files.download(latest)
```

```
import shutil

# 1. create the Drive folder (this is the part that didn't exist)
os.makedirs("/content/drive/MyDrive/shelf_project", exist_ok=True)

# 2. find the best.pt your training produced
src = max(glob.glob("/content/runs/detect/**/weights/best.pt", recursive=True),
          key=os.path.getmtime)
print("Found weights at:", src)

# 3. copy it into Drive so it survives future sessions
dst = "/content/drive/MyDrive/shelf_project/best.pt"
shutil.copy(src, dst)
print("Saved to:", dst)

# 4. now this works
model = YOLO(dst)
print("Loaded ✓")
```

8. Try it with my own data!

Pictures taken from the stores already! Let's try it with some fresh data.

1. Make a photo folder in Drive and upload into it
2. Call the shelf_report function from step 6 (the single-class version) and run it on every photo
3. See the boxes on my photos

```
# 1. Make a photo folder in Drive and upload into

!pip install pillow-heif -q

from PIL import Image
import pillow_heif

pillow_heif.register_heif_opener()      # teaches PIL to open .HEIC

photo_dir = "/content/drive/MyDrive/shelf_project/photos"
os.makedirs(photo_dir, exist_ok=True)

heic_files = glob.glob("/content/*.HEIC") + glob.glob("/content/*.heic")
print("Found", len(heic_files), "HEIC files")

for path in heic_files:
    img = Image.open(path).convert("RGB")
    name = os.path.splitext(os.path.basename(path))[0] + ".jpg"
    out = os.path.join(photo_dir, name)
    img.save(out, "JPEG", quality=90)
    print("Converted ->", out)
```

```
# 2. Call the shelf_report function from step 6 (the single-class version) and run it on every photo

for img in glob.glob(f"{photo_dir}/*"):
    shelf_report(img)
```

```
# 3. See the boxes on my photos
for img in glob.glob(f"{photo_dir}/*"):
    r = model.predict(img, conf=0.25, verbose=False)[0]
    display(Image.fromarray(r.plot()[..., :-1])) # photo with boxes drawn
```

Results: Save it and upload it to Github

```
# 1. Save the annotated photos as files
results_dir = "/content/drive/MyDrive/shelf_project/results"
os.makedirs(results_dir, exist_ok=True)

for img in glob.glob(f"{photo_dir}/*.jpg"):
    r = model.predict(img, conf=0.25, verbose=False)[0]
    out = os.path.join(results_dir, "pred_" + os.path.basename(img))
```

```
Image.fromarray(r.plot()[..., ::-1]).save(out)  
print("Saved", out)
```